

Módulo 1: Sistema Operativo y Gestión de Archivos

Propósito

Aprenderás a automatizar operaciones de archivos desde cero, empezando por observar el sistema y terminando con un organizador seguro de carpetas.

Nivel: principiante absoluto en automatización.

Duración sugerida: 6 sesiones de 90 minutos.

Producto final: organizador de archivos con modo de prueba, reglas de clasificación, protección contra sobrescritura y registro de movimientos.

Ruta de aprendizaje

Ruta -> Archivo -> Carpeta -> Clasificación -> Plan -> Ejecución segura -> Registro

Sesión	Tema	Evidencia
1	Qué es automatizar y cómo funciona una ruta	inspección de una carpeta temporal
2	pathlib: crear, leer y describir archivos	inventario de archivos
3	os: entorno, directorio actual y compatibilidad	script de diagnóstico
4	shutil: copiar, mover y proteger destinos	copia controlada
5	Diseño seguro: simulación, validación y logs	plan de movimientos
6	Proyecto: organizador por extensión	script final y demostración

1. ¿Qué es automatización?

Automatizar es convertir una tarea repetitiva en una secuencia de instrucciones que una computadora puede ejecutar de forma consistente.

Antes de programar

Describe la tarea en lenguaje humano:

1. elegir una carpeta de entrada;
2. revisar cada archivo;
3. identificar una regla, por ejemplo la extensión;
4. decidir una carpeta de destino;
5. comprobar que no se perderá información;
6. ejecutar y registrar el resultado.

Caso de uso

Una persona recibe diariamente PDFs, imágenes, hojas de cálculo y archivos de texto en Descargas. El organizador puede ahorrar tiempo, pero solo debe operar sobre una carpeta confirmada y permitir revisar el plan antes de mover nada.

2. Rutas y sistema de archivos

Una ruta describe dónde está un archivo o una carpeta.

- ruta absoluta: comienza desde una raíz, por ejemplo C:/Usuarios/Ana/Descargas;
- ruta relativa: depende del directorio desde el que se ejecuta el programa, por ejemplo datos/informes;
- directorio actual: carpeta desde la que Python interpreta las rutas relativas.

Usa `pathlib.Path` en lugar de unir cadenas manualmente:

```
from pathlib import Path

base = Path("datos")
archivo = base / "informe.csv"
print(archivo)
```

La operación `/` construye rutas adaptadas al sistema operativo.

Inspección básica

```
archivo.exists()
archivo.is_file()
archivo.parent
archivo.name
archivo.suffix
archivo.stem
```

Práctica 1: crea una ruta para `proyecto/datos/ventas.csv` y escribe qué devuelve cada propiedad.

3. Crear, leer y escribir archivos

```
from pathlib import Path

carpeta = Path("zona_prueba")
carpeta.mkdir(exist_ok=True)

nota = carpeta / "nota.txt"
nota.write_text("Primera automatización", encoding="utf-8")
contenido = nota.read_text(encoding="utf-8")
print(contenido)
```

`write_text` reemplaza el contenido existente. Para conservar información, usa `open` con modo de anexo y comprende primero la tarea.

```
with nota.open("a", encoding="utf-8") as archivo:
    archivo.write("\nSegunda línea")
```

Error frecuente: usar una ruta relativa suponiendo que siempre parte de la carpeta del script. Comprueba `Path.cwd()` o usa una ruta construida desde una ubicación conocida.

4. Listar una carpeta

```
for elemento in sorted(carpeta.iterdir()):  
    tipo = "archivo" if elemento.is_file() else "carpeta"  
    print(tipo, elemento.name)
```

`iterdir()` muestra los elementos inmediatos. No recorre subcarpetas. Esa limitación es útil para la primera versión del organizador porque reduce el riesgo de mover archivos inesperados.

Práctica 2: muestra solo los archivos `.txt` de una carpeta y cuenta cuántos hay.

5. Extensiones y reglas de clasificación

```
def extension_de(archivo: Path) -> str:
    """Devuelve una extensión normalizada o una categoría sin_extensión."""
    return archivo.suffix.lower().lstrip(".") or "sin_extensión"
```

Una regla simple puede asignar una carpeta por extensión:

```
extension = extension_de(Path("foto.JPG"))
destino = Path("organizado") / extension
```

Una regla más útil agrupa extensiones relacionadas:

```
CATEGORIAS = {
    "imagenes": {"jpg", "jpeg", "png", "gif"},
    "documentos": {"pdf", "docx", "txt"},
    "datos": {"csv", "xlsx", "json"},
}
```

Desafío: implementa `categoria_para(extension)` que devuelva otros cuando no encuentre una coincidencia.

6. os: información del entorno

os permite consultar características del sistema y variables de entorno.

```
import os

print(os.name)
print(os.getcwd())
print(os.getenv("USERNAME", "desconocido"))
```

Para construir rutas, `pathlib` suele ser más legible. Usa `os` cuando necesites información del sistema, variables de entorno o compatibilidad con APIs antiguas.

Caso de uso: obtener una carpeta configurable sin escribir el nombre del usuario directamente.

7. shutil: copiar y mover

```
import shutil

origen = Path("entrada.txt")
copia = Path("respaldo") / origen.name
shutil.copy2(origen, copia)
```

Copiar conserva el original. Mover cambia su ubicación:

```
shutil.move(str(origen), str(destino))
```

Antes de mover:

- confirma que el origen es un archivo;
- crea la carpeta de destino solo cuando corresponde;
- comprueba si el destino ya existe;
- genera un nombre alternativo o detén la operación;
- registra origen y destino.

8. Diseño seguro: planificar antes de ejecutar

Separa la decisión de la acción:

```
descubrir -> clasificar -> planificar -> revisar -> ejecutar -> verificar
```

El modo de prueba debe producir un plan sin modificar archivos. Solo después de revisarlo se permite `modo_prueba=False`.

```
def mover_seguro(origen: Path, destino: Path, modo_prueba: bool = True) -> None:
    """Muestra o ejecuta un movimiento, sin sobrescribir."""
    if destino.exists():
        raise FileExistsError(f"El destino ya existe: {destino}")
    if modo_prueba:
        print(f"PLAN: {origen} -> {destino}")
        return
    destino.parent.mkdir(parents=True, exist_ok=True)
    shutil.move(str(origen), str(destino))
```

9. Casos de uso

Caso A: Descargas personales

Clasificar PDFs, imágenes, documentos y datos. Requisito: no tocar subcarpetas y no sobrescribir.

Caso B: Copia de respaldo

Copiar archivos importantes a una carpeta de respaldo con fecha. Requisito: copiar antes de eliminar o mover.

Caso C: Preparar datos para análisis

Separar CSV, JSON y XLSX de archivos visuales. Requisito: producir un inventario para que otra persona pueda reproducir la organización.

Caso D: Limpieza supervisada

Detectar archivos antiguos y generar un informe de candidatos. Requisito: no eliminar automáticamente; primero mostrar el plan.

10. Desafíos graduados

1. Explorador: lista archivos y muestra nombre, extensión y tamaño.
2. Clasificador: asigna una categoría a cada extensión.
3. Planificador: genera pares de origen y destino sin mover.
4. Protector: evita colisiones con nombres alternativos.
5. Organizador: ejecuta movimientos solo tras confirmación.
6. Auditor: guarda un registro .csv con fecha, origen, destino y estado.

El dossier `03_desafios_y_casos.md` contiene instrucciones, pistas y criterios para cada nivel.

11. Entrega y evaluación

Criterio	Peso	Evidencia
Conceptos base	20%	explica rutas, archivos y extensiones
Descomposición	20%	separa descubrir, clasificar y ejecutar
Implementación	25%	organiza correctamente archivos de prueba
Seguridad	25%	modo de prueba, no sobrescritura y validaciones
Comunicación	10%	registro y explicación de decisiones

Antes de usar una carpeta real

- ☐ Probé con una carpeta temporal.
- ☐ Revisé el plan completo.
- ☐ El modo de prueba no modifica archivos.
- ☐ El destino no sobrescribe archivos.
- ☐ Sé cómo detener el script.
- ☐ Tengo una copia de respaldo si la operación es importante.